

CS 7643 Project Proposal

Jakob Loedding, Cristian Gutu, Hung Yu Hsiao

Team Name: Team Finance

Project Title: Learning Buy/Sell/Hold Signals for Stock Trading Using Deep Neural Networks

Project summary:

This project aims to develop a deep learning model that predicts buy, sell, or hold signals for SPY (ETF tracking a market cap-weighted index of US large- and mid-cap stocks selected by the S&P Committee) using historical market data. We will use a combination of technical indicators to decide on the market action instead of directly trying to predict future raw price – fundamentally turning this into a classification problem based on future returns. The model's predictions will be used to simulate a trading strategy over time, starting with a fixed amount of capital.

The core focus of the project is to explore and understand if a neural network can be used to learn trading patterns that would outperform a simpler buy-and-hold strategy (our benchmark). This problem is challenging due to the noisy and time-dependent nature of financial data, making it well-suited for deep learning methods. Additionally, the fact that we have detailed historical data makes this project practical because we can clearly evaluate it through back testing.

What you will do:

We will use historical daily stock data and compute technical indicators such as moving averages, RSI, MACD, and Bollinger Bands (we may decide to change these or add additional based on the observed effectiveness). These features will be organized into rolling windows and used as inputs to our neural network model. Initially, we are planning to build an LSTM model (but may also compare with MLP and 1D CNN if time permits). The model will output buy, sell, or hold decisions based on future return thresholds.

We will simulate trading using an all-in strategy, where the model either fully invests available cash, fully sells all shares, or does nothing at each time step. As a baseline, we will use a buy-and-hold strategy that invests all capital at the beginning and sells all shares at the final time step. We will then compare final portfolio values against this baseline. As part of this comparison, we may consider additional financial metrics like Sharpe ratio, maximum downturn, win rate, and profit factor. We will also investigate the following loss functions.

Approach 1: Categorical Cross-Entropy

For the first approach, we will optimize for statistical accuracy. We will act as the teacher by providing the model with pre-calculated "correct answers" (labels generated via the Triple-Barrier method) and use standard cross-entropy loss. The model simply learns to guess the right category. This makes it easier to build and highly stable; however, it optimizes being "right" rather than strictly profitable.

Approach 2: Custom (Sharpe or Return Loss)

For the second approach, we will optimize for financial performance. We will remove the labels completely and map the model's raw probabilities directly into a continuous trade size. By using a custom differentiable loss function (like the Negative Sharpe Ratio), we will force the neural network to mathematically care only about maximizing risk-adjusted returns. This requires larger data batches and is much harder to stabilize during training.

Resources / Related Work:

There is significant prior work that explores the same question as we've posed – whether a neural network model can be used to generate effective trade signals. This area has been especially explored with respect to using LSTMs and CNNs for this task [4], [5]. Some survey papers have shown that deep neural networks are widely used for stock forecasting, but their performance depends on various factors (like feature engineering) [1]. As a result, we will place an emphasis on the combination of technical indicators we will use and how they can complement each other to capture various market conditions [3].

More recently, some state-of-the-art systems have leveraged a hybrid approach by combining both CNNs and LSTMs [2]. Additionally, there are models that embed features like sentiment signals (based on news) which have shown to also influence the market. Labeling techniques such as the Triple Barrier Method [5] further help create more robust training signals. Approaches that directly optimize financial objectives like the Sharpe ratio [6] also demonstrate the importance of aligning model design with real-world trading performance.

[1] "Stock Market Prediction via Deep Learning Techniques", Zou et al.

[2] "A stock market trading framework based on deep learning and technical analysis", Shah et al.

[3] "Key technical indicators for stock market prediction", Mostafavi et al.

[4] "High-performance stock index trading: making effective use of a deep LSTM neural network", arXiv preprint arXiv:1902.03125.

[5] "Advances in Financial Machine Learning.", M. López de Prado

[6] “Optimization using financial metrics,” arXiv:1904.04912.

Dataset:

We will be using the yfinance Python API (<https://github.com/ranaroussi/yfinance>) to obtain historical open, high, low, close, and volume data (to be used in calculating technical indicators and the associated “buy”, “sell”, “hold” labels). Labels will be generated using the Triple Barrier Method, where each timestep is assigned a buy, sell, or hold signal depending on whether future price movements reach predefined upper (profit-taking) or lower (stop-loss) barriers within a fixed time horizon; otherwise, the label is hold. We have crawled 10 years of SPY price data from 2005 to 2025. Below is a link to the crawled data:

https://github.com/gutucristian/CS7643-Final-Project/blob/main/SPY_ohlcv.csv